Software Requirements Specification

for

CampusFix — Campus Technical Issue and Resource Management Portal

Version 1.0 approved

Prepared by
Shreya Gupta - [202401100200295]
Shraddha Singh – [202401100200294]
Vatsala Sahu – [202401100200342]
Yashi Bansal – [202401100200361]

Department of Computer Science & Engineering
KIET Group of Institutions, Ghaziabad
Batch 2024–2028
September 2026

Table of Contents

Table of Contents 2
Revision History 5
1. Introduction 6
1.1 Purpose 6
1.2 Document Conventions 6
1.3 Intended Audience and Reading Suggestions 6
1.4 Product Scope 6
1.5 References 7
2. Overall Description 8
2.1 Product Perspective 8
2.2 Product Functions 8
2.3 User Classes and Characteristics 9
2.4 Operating Environment 9
2.5 Design and Implementation Constraints 9
2.6 User Documentation 9
2.7 Assumptions and Dependencies 10
3. External Interface Requirements 11
3.1 User Interfaces 11
3.2 Hardware Interfaces 11
3.3 Software Interfaces 11
3.4 Communications Interfaces 11
4. System Features 12
4.1 User Authentication and Role-Based Access 12
4.1.1 Description and Priority 12
4.1.2 Stimulus/Response Sequences 12
4.1.3 Functional Requirements 12
4.2 Issue Reporting 12
4.2.1 Description and Priority 12
4.2.2 Stimulus/Response Sequences 12
4.2.3 Functional Requirements 12
4.3 Issue Verification and Assignment 12
4.3.1 Description and Priority 12
4.3.2 Stimulus/Response Sequences 12

4.3.3 Functional Requirements 13
4.4 Status Tracking and Notifications 13
4.4.1 Description and Priority 13
4.4.2 Stimulus/Response Sequences 13
4.4.3 Functional Requirements 13
4.5 Feedback and Reopening 13
4.5.1 Description and Priority 13
4.5.2 Stimulus/Response Sequences 13
4.5.3 Functional Requirements 13
4.6 Search, Filtering, and Master Data Management 13
4.6.1 Description and Priority 13
4.6.2 Stimulus/Response Sequences 14
4.6.3 Functional Requirements 14
4.7 Dashboard and Reporting 14
4.7.1 Description and Priority 14
4.7.2 Stimulus/Response Sequences 14
4.7.3 Functional Requirements 14
5. Other Nonfunctional Requirements 15
5.1 Performance Requirements 15
5.2 Safety Requirements 15
5.3 Security Requirements 15
5.4 Software Quality Attributes 15
5.5 Business Rules 15
6. Other Requirements 17
6.1 Testing Strategy and Test Cases 17
6.2 Risk Management 17
6.3 Priority Definitions and Issue Categories 18
6.4 SDLC Model and Development Schedule 18
6.5 Legal / Ethical Considerations 19
6.6 Planned Future Enhancements (Out of Current Scope) 19
Appendix A: Glossary 20
Appendix B: Analysis Models 21
B.1 System Modules 21
B.2 Use Case Model 21

B.3 Class Diagram 22
B.4 Sequence and State Diagrams 22
B.4.1 Sequence — Reporting an Issue 22
B.4.2 State Diagram — Issue Lifecycle 22
B.5 Data Flow Diagrams 22
B.5.1 DFD Level 0 (Context Diagram) 22
B.5.2 DFD Level 1 23
B.6 ER Diagram and Database Design 23
B.7 Feasibility Summary 23
Appendix C: To Be Determined List 25

Revision History

Name	Date	Reason for Changes	Version
Shreya	10-Sep-2026	Reorganized project documentation into IEEE 830 SRS format	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of CampusFix, a centralized, web-based portal for reporting and tracking campus technical and infrastructural issues (e.g., malfunctioning projectors, non-functional lab computers, faulty air conditioning, Wi-Fi outages, and damaged furniture). It captures the functional and non-functional requirements of the complete system and is intended to serve as the single reference for the design, development, and testing of CampusFix.

Campuses currently rely on informal channels — verbal complaints, phone calls, or scattered emails — to report such issues, which leads to delays, lost requests, and no accountability for resolution. This SRS covers the full system: issue reporting, verification and assignment, status tracking, notifications, feedback, and administrative dashboards and reports.

1.2 Document Conventions

- Each functional requirement is uniquely tagged with the prefix FR- followed by a sequence number (e.g., FR-01) for traceability to features, use cases, and test cases.
- The keyword "shall" indicates a mandatory requirement; "should" indicates a recommended but non-mandatory capability.
- Priorities for system features are expressed qualitatively as High, Medium, or Low; unless stated otherwise, a feature's priority is inherited by all functional requirements listed under it.
- Use case identifiers use the prefix UC- (e.g., UC-01); test case identifiers use the prefix TC-.

1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team, project guide/mentor and evaluators, and, at a high level, campus administration stakeholders (students, faculty, administrators, technicians, and management) who represent the system's user classes.

Readers new to the project should begin with Section 1 (Introduction) and Section 2 (Overall Description) for context, then proceed to Section 3 and Section 4 for detailed interface and functional requirements. Section 5 covers non-functional requirements, and Section 6 and the appendices contain supporting design, testing, and planning material referenced throughout the document.

1.4 Product Scope

CampusFix replaces ad-hoc manual reporting with a structured digital workflow. Students, faculty, and staff report issues with structured details — category, location, priority, and optional photographic evidence — and track resolution through a defined lifecycle: Submitted, Acknowledged, Assigned, In Progress, Resolved, and Closed. Administrators verify and prioritize issues and assign them to technicians; technicians update status and record resolution evidence; management uses dashboards and reports for maintenance planning.

Key objectives:

- Provide a single centralized platform for reporting all categories of campus issues.
- Reduce the time taken between issue reporting and resolution.
- Improve communication between reporters, administrators, and technicians.
- Establish clear accountability through issue assignment and status tracking.
- Maintain a complete, searchable history of all reported issues.
- Allow prioritization of critical issues over minor ones.
- Provide management with dashboards and reports for maintenance planning.

In scope:

- User authentication and role-based access.
- Issue reporting with category, location, priority, and optional image evidence.
- Administrator verification and technician assignment.
- Real-time status tracking through the issue lifecycle.
- In-app/email notifications.
- Feedback collection after resolution.
- Dashboards and analytical reports.

Out of scope:

- Physically repairing equipment or performing maintenance work.
- Procurement or purchasing of replacement parts/equipment.
- Direct management or contracting of external vendors.

1.5 References

- Sommerville, I., Software Engineering, Pearson Education.
- Pressman, R.S., Software Engineering: A Practitioner's Approach, McGraw-Hill.
- Booch, G., Rumbaugh, J., Jacobson, I., The Unified Modeling Language User Guide.
- Elmasri, R., Navathe, S.B., Fundamentals of Database Systems, Pearson Education.
- OWASP Foundation, OWASP Top Ten Web Application Security Risks, owasp.org.
- Official documentation of the frontend/backend frameworks and database systems used in implementation (Node.js, Express, MongoDB).

2. Overall Description

2.1 Product Perspective

CampusFix is a new, self-contained web application; it is not a follow-on to an existing system. It is designed as a role-based application supporting distinct workflows for reporters (students, faculty/staff), administrators, technicians, and management, built around a single, well-understood issue lifecycle.

The system follows a three-tier architecture:

- Presentation Layer — the user interface through which students, faculty, administrators, and technicians interact with the system (issue reporting forms, dashboards, status trackers).
- Application / Business Logic Layer — handles authentication, authorization, issue lifecycle rules, notification triggers, priority and escalation logic, and coordination between the presentation and data layers.
- Database Layer — stores all persistent data (users, issues, categories, locations, assignments, notifications, attachments, status history) and enforces referential integrity between related records.

A user action (e.g., submitting an issue) flows from the presentation layer to the application layer, where it is validated and processed according to business rules, before being persisted in the database layer. Responses, including status updates and notifications, flow back through the same layers to reach the user. End-to-end, the system moves through the following stages: login → issue submission → administrator verification and assignment → technician notification and status updates → resolution and evidence upload → administrator verification → user notification, feedback, or reopening → closure and archival.

2.2 Product Functions

At a high level, CampusFix must let users:

- Register and log in with role-based access.
- Submit a new issue with description, category, location, priority, and optional image evidence.
- Receive automatic notifications on submission, assignment, status change, and resolution.
- Allow administrators to verify, prioritize, and assign issues to technicians.
- Allow technicians to accept, work on, and update the status of assigned issues.
- Track an issue's status through its full lifecycle, with complete history.
- Submit feedback after resolution, or reopen an unsatisfactory resolution.
- Search and filter issues by category, location, status, and priority.
- Manage master data for categories and locations.
- View dashboards and generate reports on resolution time, technician workload, and recurring issues.

Detailed functional requirements for each of these functions are given in Section 4, System Features.

2.3 User Classes and Characteristics

Role	Key Capabilities
Student	Report issues, upload evidence, track own issues, receive notifications, give feedback.
Faculty / Staff	Report and track issues relevant to their classrooms or departments.
Administrator	Verify issues, set priority, assign technicians, monitor progress, manage users/categories/locations, generate reports.
Technician	View assigned issues, update status, add remarks, upload resolution evidence.
Management	View dashboards and analytics, monitor performance, plan maintenance decisions.

Students and Faculty/Staff are the most frequent users (issue reporters) and require the simplest, lowest-friction interface. Administrators and Technicians are moderate-frequency users who need more structured, task-oriented views. Management uses the system least frequently but requires accurate, aggregated reporting.

2.4 Operating Environment

CampusFix is built entirely with widely available, well-documented web technologies: a JavaScript/React-style frontend, a Node.js/Express backend, and a MongoDB (document) database, all of which are commonly taught and used in academic environments. The application runs as a standard web app, accessible through any modern desktop or mobile browser, and is hosted on a cloud or free-tier hosting service with a cloud-hosted MongoDB Atlas instance. No specialized or proprietary infrastructure is required.

2.5 Design and Implementation Constraints

- Development is constrained to open-source frameworks and free-tier cloud/database services to keep cost negligible for an academic project.
- The system must be deliverable within a single-semester academic project timeline (see Appendix B, Section B.7 — SDLC Model and Development Schedule).
- An Agile/Incremental development approach is required, since requirements are expected to evolve as feedback is gathered from students, administrators, and maintenance staff during early usage.
- Role-based access control, password hashing, and validated file uploads must be built in from the outset (see Section 5.3, Security Requirements).
- Image evidence uploads must be restricted by file type and size.

2.6 User Documentation

CampusFix will be accompanied by a short in-app help/FAQ section covering the issue-reporting workflow and role-specific dashboards, along with a user guide (delivered as part of the project documentation) describing how to use each role's interface. No separate printed manual is planned for the initial release.

2.7 Assumptions and Dependencies

- Users have access to a device with a modern web browser and a working internet connection.
- User accounts and roles are provisioned or approved by an administrator; the system does not assume self-service escalation of privileges.
- The project depends on free-tier or academic-tier availability of the chosen hosting and database services (e.g., MongoDB Atlas); the scope and cost assumptions would need revisiting if these are withdrawn.
- The system must handle user data and uploaded images responsibly, restrict visibility of personal information to authorized roles, and include safeguards against misuse such as false or malicious issue reports.

3. External Interface Requirements

3.1 User Interfaces

CampusFix provides a responsive, form-based web interface. Key screens include: an issue-reporting form (description, category, location, priority, optional image), role-specific dashboards (student/faculty, administrator, technician, management), a status-tracking view showing an issue's current lifecycle stage and history, and administrative screens for verifying, assigning, and managing master data (categories, locations). Every screen follows a consistent layout with clear validation error messaging, so that non-technical students and staff can use the system without training.

3.2 Hardware Interfaces

No dedicated hardware is required. The application runs on any standard desktop, laptop, tablet, or smartphone capable of running a modern web browser. A device camera or file system access is used only indirectly, to select or capture an image when attaching evidence to an issue.

3.3 Software Interfaces

- Frontend: JavaScript/React-based single-page application communicating with the backend over HTTP(S)/REST.
- Backend: Node.js with Express, exposing REST APIs for authentication, issue management, assignment, notifications, and reporting.
- Database: MongoDB (via MongoDB Atlas), storing Users, Issues, Categories, Locations, Assignments, StatusHistory, Notifications, Attachments, and Feedback collections.
- Notification service: an email (and optionally SMS/in-app) delivery mechanism triggered by backend events such as issue submission, assignment, status change, and resolution.
- Authentication: JSON Web Tokens (JWT) used for session/authorization between frontend and backend.

3.4 Communications Interfaces

All client–server communication uses HTTPS. The system sends in-app and/or email notifications on issue submission (FR-04), and on assignment, status change, and resolution (FR-08). Message formats for API requests/responses are JSON. No special data-transfer-rate or synchronization requirements apply beyond standard web request/response latency.

4. System Features

Functional requirements are organized below by system feature. Each functional requirement retains its original FR-xx identifier for traceability to the use cases (Appendix B) and test cases (Section 6.1).

4.1 User Authentication and Role-Based Access

4.1.1 Description and Priority

Allows all users to register and log in, and restricts visible features and actions according to the user's assigned role (Student, Faculty/Staff, Administrator, Technician, Management). Priority: High.

4.1.2 Stimulus/Response Sequences

User submits login credentials → system validates and authenticates → user is redirected to a role-specific dashboard. Invalid credentials → system displays an authentication error and blocks access.

4.1.3 Functional Requirements

- FR-01: The system shall allow users to register and log in with role-based access.

4.2 Issue Reporting

4.2.1 Description and Priority

Allows Students and Faculty/Staff to submit a new campus issue with structured details and optional photographic evidence. Priority: High.

4.2.2 Stimulus/Response Sequences

User opens "Report Issue" → enters description, category, location, priority → optionally attaches an image → submits → system creates the issue with status "Submitted" and notifies the administrator. Missing required fields → system displays a validation error and blocks submission.

4.2.3 Functional Requirements

- FR-02: Users shall be able to submit a new issue with description, category, location, and priority.
- FR-03: Users shall be able to attach an optional image as evidence when reporting an issue.

4.3 Issue Verification and Assignment

4.3.1 Description and Priority

Notifies administrators of new issues and allows them to verify, prioritize, and assign issues to technicians. Priority: High.

4.3.2 Stimulus/Response Sequences

New issue submitted → administrator is notified → administrator opens the issue, confirms/adjusts priority, and selects an available technician → issue status changes to "Assigned" and the technician is notified. If no technician is available, the issue remains "Acknowledged" and may be escalated.

4.3.3 Functional Requirements

- FR-04: The system shall notify the concerned administrator upon new issue submission.
- FR-05: Administrators shall be able to verify, prioritize, and assign issues to technicians.

4.4 Status Tracking and Notifications

4.4.1 Description and Priority

Lets technicians update assigned issues and keeps all stakeholders informed of status changes, while the system maintains a full audit trail. Priority: High.

4.4.2 Stimulus/Response Sequences

Technician opens an assigned issue and updates its status (e.g., to "In Progress") → status history is updated and visible to the reporter → a notification is sent to the reporter on assignment, status change, and resolution.

4.4.3 Functional Requirements

- FR-06: Technicians shall be able to view, accept, and update the status of assigned issues.
- FR-07: The system shall track and display each issue's current status through its lifecycle.
- FR-08: The system shall send notifications on assignment, status change, and resolution.
- FR-13: The system shall maintain a complete status-change history for every issue.

4.5 Feedback and Reopening

4.5.1 Description and Priority

Allows reporters to close the loop on a resolved issue by rating the resolution or reopening it if unsatisfied. Priority: Medium.

4.5.2 Stimulus/Response Sequences

Issue marked "Resolved" → reporter is notified → reporter submits feedback (recorded and linked to the issue) or reopens the issue → status reverts to "Assigned"/"Reopened".

4.5.3 Functional Requirements

- FR-09: Users shall be able to submit feedback after an issue is marked resolved.
- FR-10: Users shall be able to reopen an issue if unsatisfied with the resolution.

4.6 Search, Filtering, and Master Data Management

4.6.1 Description and Priority

Lets Administrators and Technicians locate relevant issues quickly and lets Administrators maintain the category and location reference data used throughout the system. Priority: Medium.

4.6.2 Stimulus/Response Sequences

User applies filters for category, location, status, and/or priority → only matching issues are displayed. Administrator adds/edits a category or location → master data is updated system-wide.

4.6.3 Functional Requirements

- FR-11: The system shall allow searching and filtering of issues by category, location, status, and priority.
- FR-12: Administrators shall be able to manage master data for categories and locations.

4.7 Dashboard and Reporting

4.7.1 Description and Priority

Provides Administrators and Management with summarized, role-specific views of issue counts and generated reports for maintenance planning. Priority: Medium.

4.7.2 Stimulus/Response Sequences

Administrator/Management opens the dashboard → system displays correct counts of open/in-progress/resolved issues. User requests a resolution-time report → system generates a report reflecting accurate average resolution time, technician workload, and recurring issues.

4.7.3 Functional Requirements

- FR-14: The system shall provide a dashboard summarizing open, in-progress, and resolved issues.
- FR-15: The system shall generate reports on resolution time, technician workload, and recurring issues.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Issue submission and dashboard pages shall load within 2–3 seconds under normal load.
- The system shall support a growing number of users, issues, and categories over time without degradation (scalability).

5.2 Safety Requirements

CampusFix does not directly control physical equipment and has no safety-critical hardware interfaces. The main safety-adjacent concern is ensuring that issues flagged as Critical (e.g., electrical faults, water leakage near equipment) are visibly prioritized and promptly routed to administrators, so that genuine hazards are not obscured by lower-priority reports (see Section 6.3, Priority Definitions).

5.3 Security Requirements

- Role-Based Access Control (RBAC) restricting features by user role.
- Secure password storage using industry-standard hashing.
- Server-side validation of all form inputs to prevent injection attacks.
- Restricted file-upload types and size limits for evidence images.
- Session management with secure, expiring tokens (JWT).
- Audit trail of status changes and assignments for accountability.
- Access to personal and uploaded data limited to authorized roles.

5.4 Software Quality Attributes

Attribute	Description as applied to CampusFix
Reliability	The system should correctly persist issue data and status transitions without loss.
Availability	The portal should be accessible to users during working hours with minimal downtime.
Usability	The interface should be simple enough for non-technical students and staff to use without training.
Maintainability	Modular design should allow features to be updated or extended with minimal impact elsewhere.
Data Integrity	Referential integrity must be preserved between issues, users, assignments, and history records.
Privacy	Uploaded images and personal data must only be visible to authorized roles.

5.5 Business Rules

- Only Administrators may verify an issue, set/adjust its priority, and assign it to a technician.

- Only the Technician to whom an issue is assigned may update that issue's status or upload resolution evidence for it.
- Only the original reporter may submit feedback for, or request reopening of, their own resolved issue.
- Only Administrators may create or edit master data (categories and locations).
- Dashboards and cross-issue reports are visible only to Administrator and Management roles.

6. Other Requirements

6.1 Testing Strategy and Test Cases

CampusFix testing follows a layered strategy: Unit Testing of individual functions/components, Integration Testing of module interactions (e.g., issue creation triggering notifications), System Testing of the complete application, and User Acceptance Testing with representative students, staff, and technicians. Regression, performance, security, and usability testing are performed prior to each release.

TC ID	Req. ID	Scenario	Expected Result
TC-01	FR-01	Login with valid credentials	User is authenticated and redirected to their dashboard.
TC-02	FR-01	Login with invalid credentials	System displays an authentication error.
TC-03	FR-02	Submit issue with all required fields	Issue is created with status 'Submitted'.
TC-04	FR-02	Submit issue with missing description	System shows validation error, blocks submission.
TC-05	FR-03	Upload a supported image format as evidence	Image is attached and linked to the issue.
TC-06	FR-03	Upload an unsupported/oversized file	System rejects the file with an error message.
TC-07	FR-05	Administrator assigns a technician to an issue	Issue status changes to 'Assigned'; technician notified.
TC-08	FR-06	Technician updates status to 'In Progress'	Status history is updated and visible to reporter.
TC-09	FR-08	Issue status changes	Reporter receives a notification.
TC-10	FR-09	User submits feedback after resolution	Feedback is recorded and linked to the issue.
TC-11	FR-10	User reopens a resolved issue	Issue status reverts to 'Assigned'/'Reopened'.
TC-12	FR-11	Filter issues by category and status	Only matching issues are displayed.
TC-13	—	Non-admin user attempts to access admin dashboard	Access is denied (unauthorized).
TC-14	FR-14	Administrator opens dashboard	Correct counts of open/in-progress/resolved issues are shown.
TC-15	FR-15	Generate resolution-time report	Report reflects accurate average resolution time.

6.2 Risk Management

Risk	Impact	Mitigation

Risk	Impact	Mitigation
Server downtime	High	Use reliable hosting with monitoring and automated restarts.
Database failure/data loss	High	Regular automated backups and transaction logging.
Notification delivery failure	Medium	Retry mechanism and in-app fallback notifications.
Security breach / unauthorized access	High	Role-based access control, input validation, secure authentication.
Fake or duplicate issue reports	Medium	Duplicate detection prompts and administrator review before assignment.
Technician delays	Medium	SLA-based escalation to higher-level administrators.
High system load during peak reporting periods	Medium	Load testing and scalable hosting configuration.
Incorrect issue categorization by users	Low	Administrator review step before assignment.

6.3 Priority Definitions and Issue Categories

Priority	Definition
Critical	Safety hazard or major operational impact (e.g., electrical fault, water leakage near equipment).
High	Major disruption affecting classes, labs, or a large number of users (e.g., lab network down).
Medium	Moderate impact with a possible workaround (e.g., one non-functional workstation in a lab).
Low	Minor inconvenience with negligible operational impact (e.g., a loose chair leg).

Issue categories include: Electrical, Networking, Computer/Lab, HVAC, Classroom Equipment, Software, Cafeteria, Library, Plumbing, Furniture, Infrastructure, and Other. Categorization enables routing of issues to appropriately skilled technicians and supports trend analysis in reporting.

6.4 SDLC Model and Development Schedule

Among Waterfall, Agile, Spiral, and Incremental models, an Agile/Incremental approach is used for CampusFix, since requirements are likely to evolve as feedback is gathered from students, administrators, and maintenance staff during early usage. Development proceeds in short iterations, each delivering a working increment (e.g., issue reporting first, then assignment and notifications, then dashboards and reporting), allowing continuous refinement based on stakeholder feedback.

Phase	Approx. Duration
Requirement Analysis & System Study	Week 1–2
System & Database Design	Week 3–4

Phase	Approx. Duration
Core Module Development (Auth, Issue Reporting)	Week 5–7
Assignment, Notification & Status Modules	Week 8–9
Dashboard & Reporting Module	Week 10–11
Integration & System Testing	Week 12–13
User Acceptance Testing & Bug Fixing	Week 14
Deployment & Documentation	Week 15–16

6.5 Legal / Ethical Considerations

The system must handle user data and uploaded images responsibly, restrict visibility of personal information to authorized roles, and include safeguards against misuse such as false or malicious issue reports.

6.6 Planned Future Enhancements (Out of Current Scope)

- Dedicated mobile application for faster on-the-go reporting.
- AI-based automatic issue classification from the description text.
- AI-based priority prediction and duplicate-issue detection.
- Predictive maintenance using historical resolution data.
- QR codes on equipment for instant issue reporting.
- Integration with the college ERP system and WhatsApp/SMS notifications.

Appendix A: Glossary

Term	Definition
Issue	A reported campus technical/infrastructural problem tracked by the system from submission to closure.
Issue Lifecycle	The defined sequence of states an issue passes through: Submitted, Acknowledged, Assigned, In Progress, Resolved, Closed (with Rejected, Cancelled, Escalated, and Reopened as alternate/branch states).
FR	Functional Requirement — a capability the system must provide, identified as FR-xx.
NFR	Non-Functional Requirement — a quality attribute or constraint the system must satisfy (performance, security, usability, etc.).
RBAC	Role-Based Access Control — restricting system features/actions based on a user's assigned role.
Reporter	A Student or Faculty/Staff member who submits an issue.
Technician	A user responsible for resolving assigned issues and recording resolution evidence.
3NF	Third Normal Form — a database normalization level used in the CampusFix schema design.
TC	Test Case — a specific scenario used to verify a requirement, identified as TC-xx.

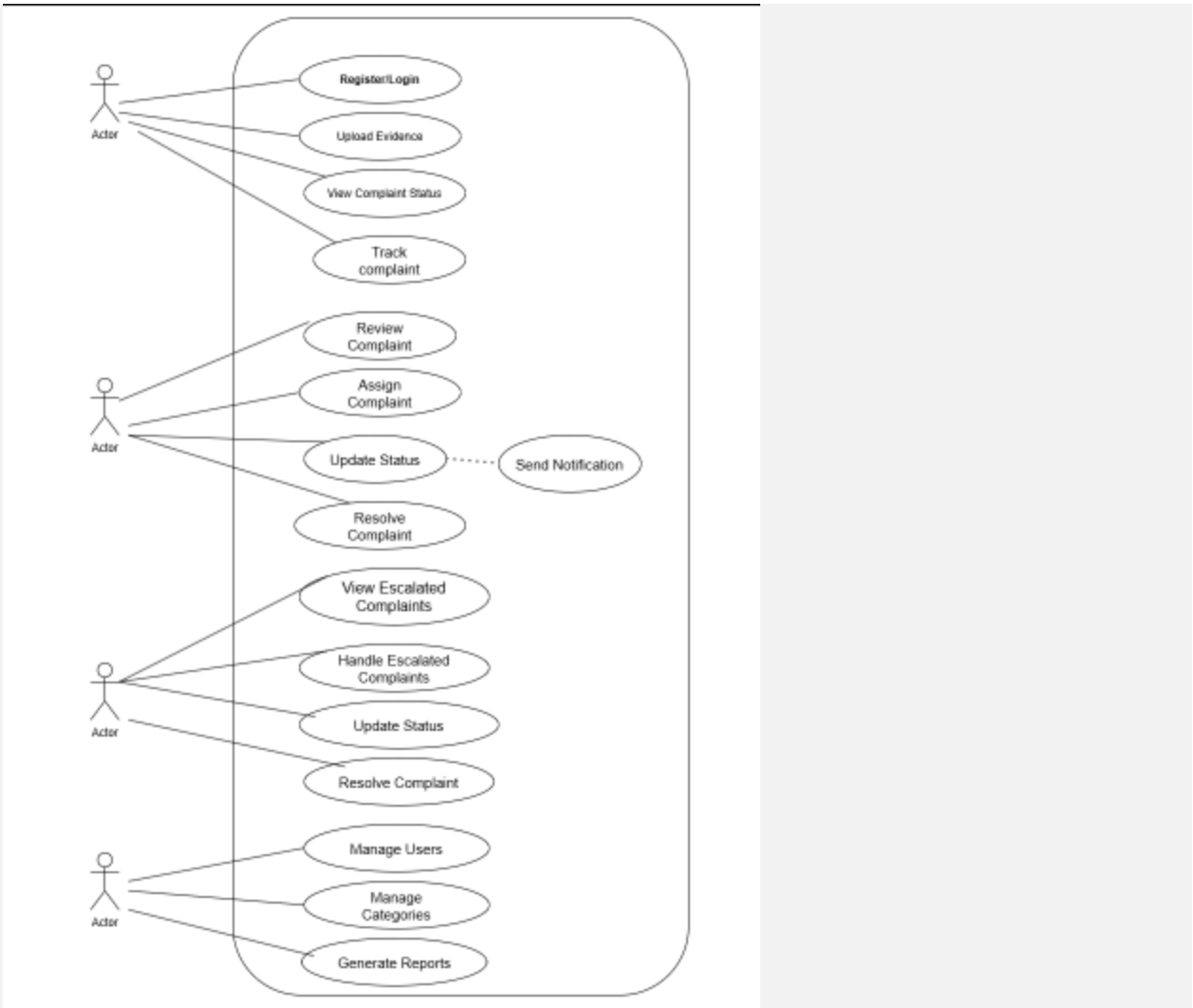
Appendix B: Analysis Models

B.1 System Modules

Module	Purpose
Authentication Module	Handles login, registration, and role-based access control for all user types.
Issue Reporting Module	Allows users to submit new issues with description, category, location, priority, and optional image evidence.
Issue Management Module	Enables administrators to review, verify, and prioritize submitted issues.
Assignment Module	Assigns verified issues to available technicians and tracks assignment history.
Status Tracking Module	Maintains and displays the current lifecycle state of each issue to all relevant users.
Notification Module	Generates in-app and/or email notifications for key events (submission, assignment, status change, resolution).
Resolution Module	Allows technicians to record remarks, upload resolution evidence, and mark issues resolved.
Feedback Module	Collects user feedback and satisfaction ratings after issue resolution, and supports reopening.
Dashboard Module	Presents summarized, role-specific views of issue counts, statuses, and workloads.
Reporting & Analytics Module	Generates aggregate reports on resolution times, category trends, and recurring issues.

B.2 Use Case Model

Actors: Student, Faculty/Staff, Administrator, Technician, Management.



Use Case: Report Issue

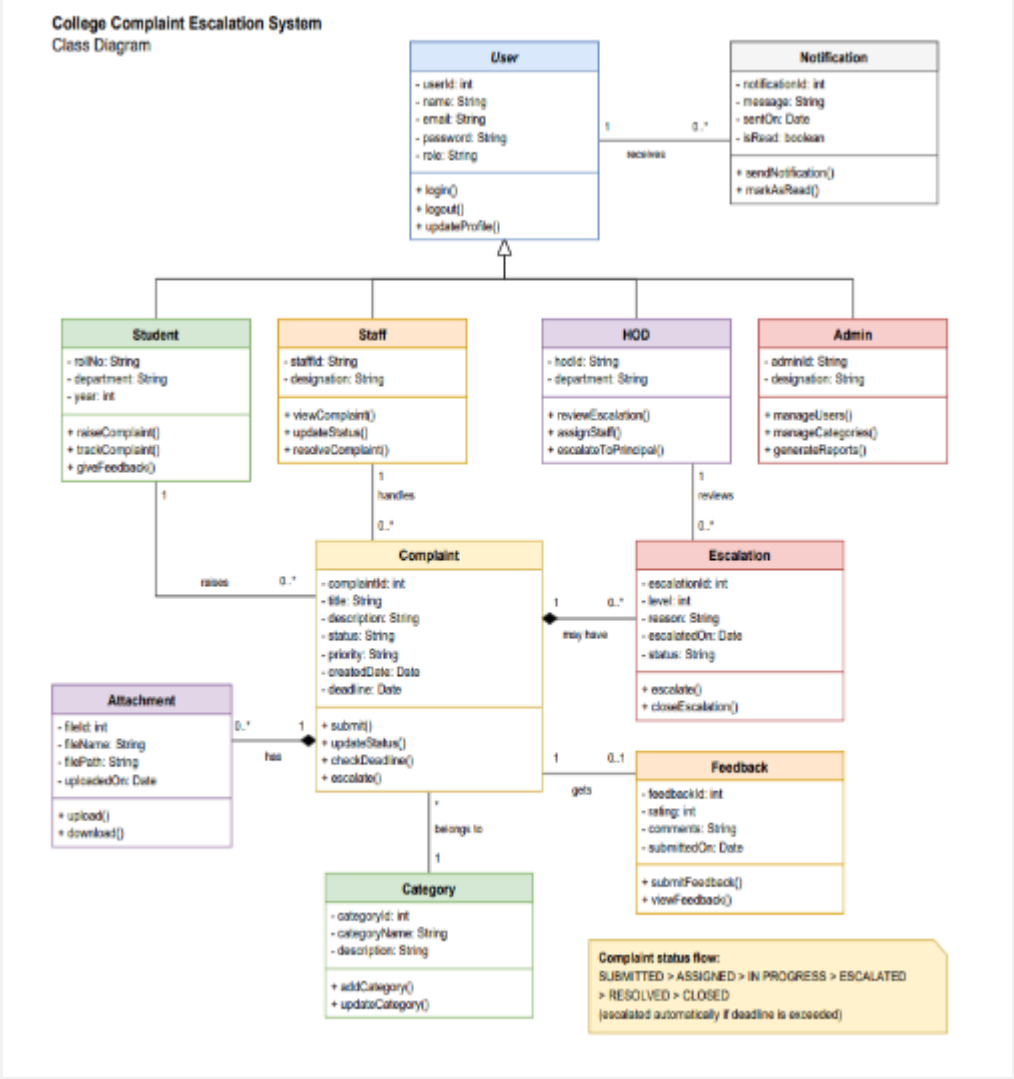
Field	Description
Use Case ID / Name	UC-01 / Report Issue
Actor	Student, Faculty/Staff
Description	Allows a user to submit a new campus issue with relevant details.
Preconditions	User is logged in.
Main Flow	User navigates to 'Report Issue' → fills description, category, location, priority → optionally attaches image → submits.
Alternative Flow	If required fields are missing, the system displays a validation error and the form is not submitted.
Postconditions	Issue is created with status 'Submitted' and administrator is notified.

Use Case: Assign Technician

Field	Description
Use Case ID / Name	UC-04 / Assign Technician
Actor	Administrator
Description	Allows the administrator to assign a verified issue to a technician.
Preconditions	Issue exists with status 'Acknowledged'.
Main Flow	Administrator opens issue → selects an available technician → confirms assignment.
Alternative Flow	If no technician is available, the issue remains 'Acknowledged' and may be escalated.
Postconditions	Issue status changes to 'Assigned'; technician is notified.

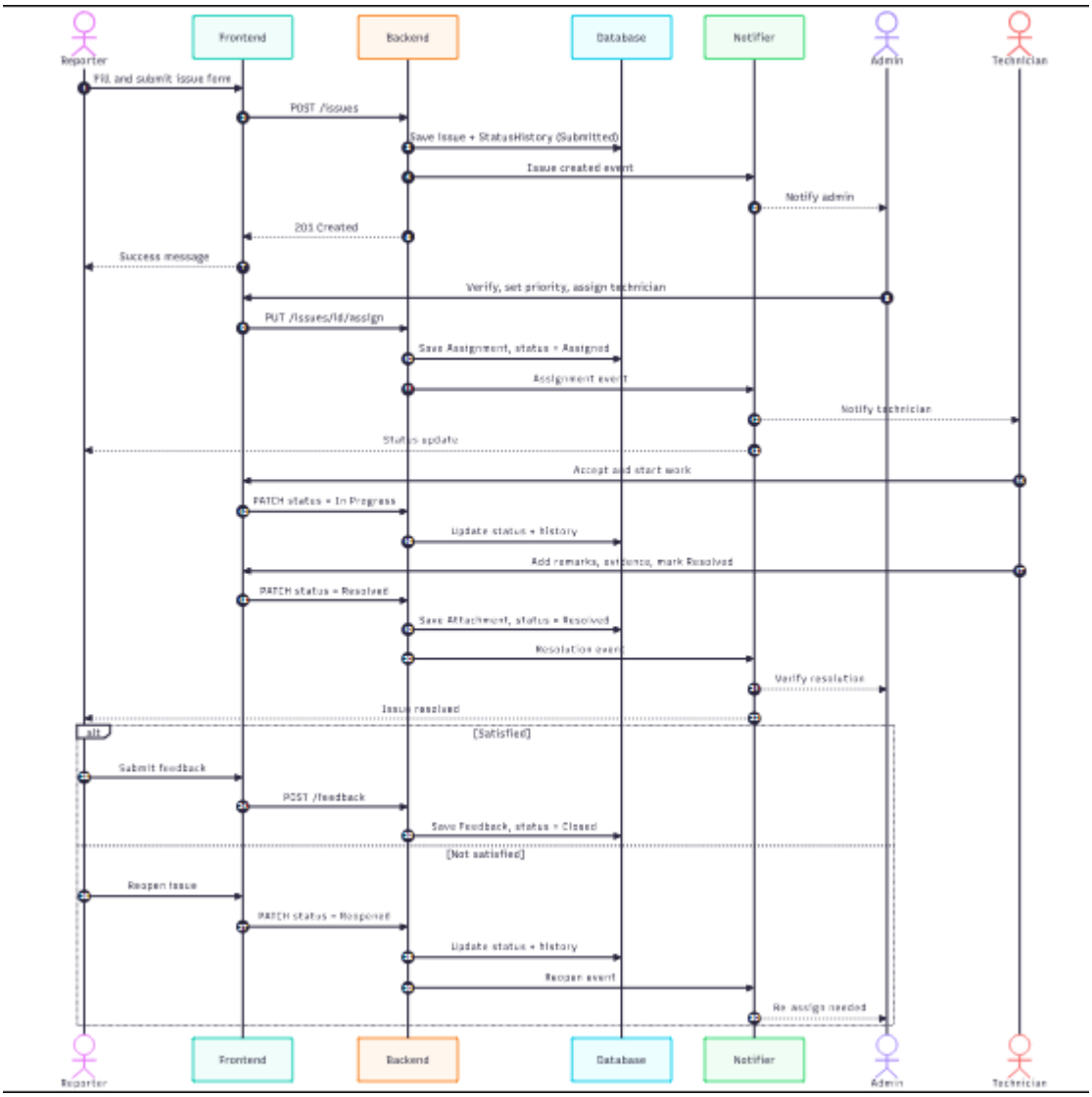
B.3 Class Diagram

- User (base class) — extended by Student, Faculty, Administrator, Technician.
- Issue (many) — reported by one User; belongs to one Category and one Location.
- Assignment — links one Issue to one Technician; records assignment timestamp.
- StatusHistory — many records per Issue, capturing every status transition.
- Notification — many records per User, linked to a triggering Issue event.
- Attachment — many per Issue (evidence images).
- Comment — many per Issue (remarks by technicians/administrators).
- Feedback — one per resolved Issue, submitted by the reporting User.

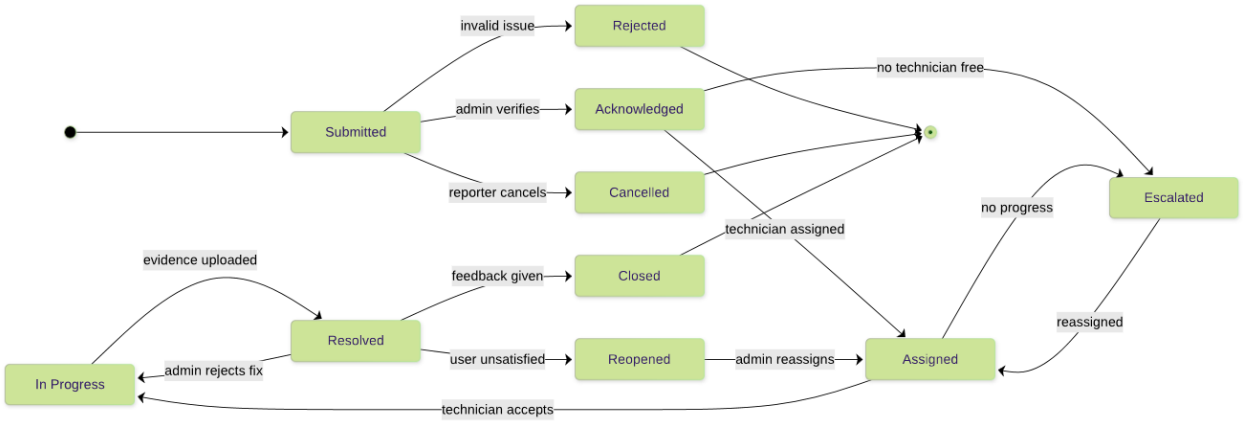


B.4 Sequence and State Diagrams

B.4.1 Sequence — Reporting an Issue



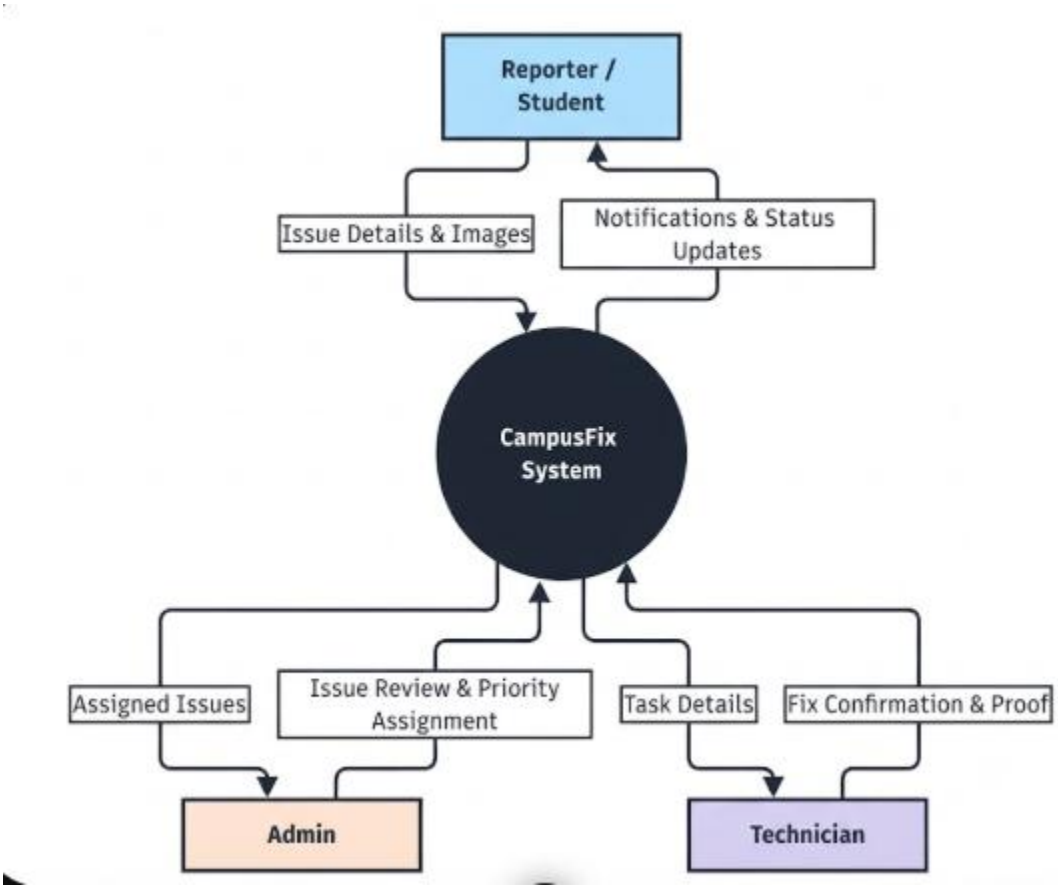
B.4.2 State Diagram — Issue Lifecycle



B.5 Data Flow Diagrams

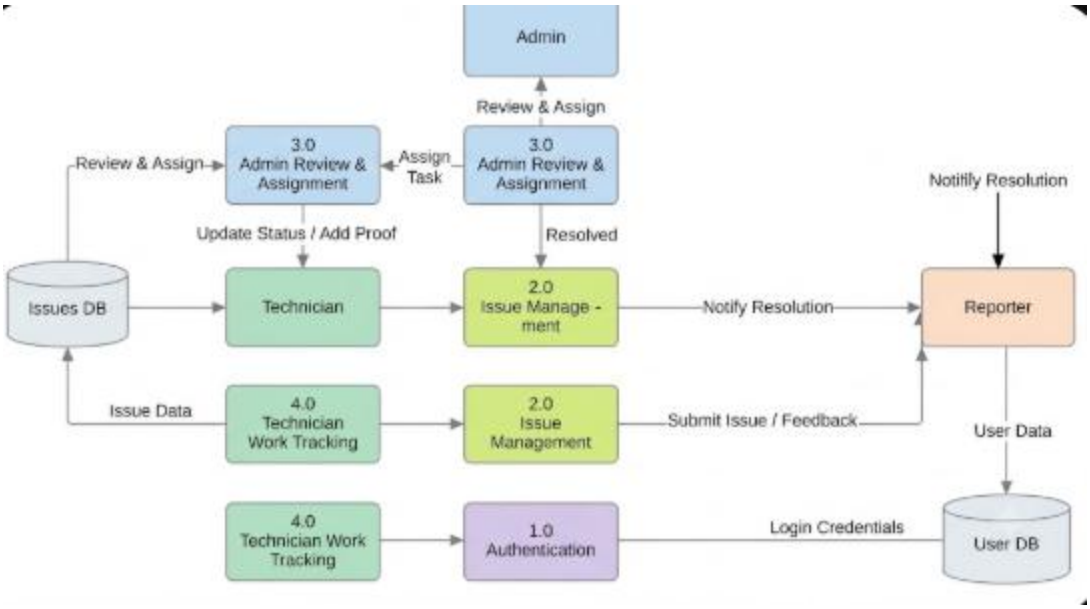
B.5.1 DFD Level 0 (Context Diagram)

External entities — Student/Faculty, Administrator, Technician, Management — interact with the central CampusFix process, which reads from and writes to the underlying data store.



B.5.2 DFD Level 1

The system decomposes into six major processes: Authentication, Issue Management, Assignment, Notification, Resolution, and Reporting — each reading and writing to its respective data stores (Users, Issues, Assignments, Notifications, StatusHistory).



B.6 ER Diagram and Database Design

Entity	Key Attributes	Relationships
User	UserID (PK), Name, Email, PasswordHash, Role	1—many with Issue, Notification
Issue	IssueID (PK), UserID (FK), CategoryID (FK), LocationID (FK), Description, Priority, Status, CreatedAt	many—1 with User, Category, Location
Category	CategoryID (PK), Name	1—many with Issue
Location	LocationID (PK), Name, Building, Floor	1—many with Issue
Assignment	AssignmentID (PK), IssueID (FK), TechnicianID (FK), AssignedAt	many—1 with Issue, Technician
StatusHistory	HistoryID (PK), IssueID (FK), Status, ChangedAt	many—1 with Issue
Notification	NotificationID (PK), UserID (FK), IssueID (FK), Message, SentAt	many—1 with User, Issue
Attachment	AttachmentID (PK), IssueID (FK), FileURL	many—1 with Issue
Feedback	FeedbackID (PK), IssueID (FK), Rating, Comments	1—1 with Issue

The schema is designed to satisfy Third Normal Form (3NF): every non-key attribute depends only on the primary key of its own entity, repeating groups (such as multiple status changes or attachments per issue) are extracted into separate related tables (StatusHistory, Attachment), and transitive dependencies are removed by referencing Category and Location as separate entities rather than embedding their details directly in the Issue table.

B.7 Feasibility Summary

- Technical: buildable entirely with widely available, well-documented web technologies (React-style frontend, Node.js/Express backend, MongoDB); no specialized or proprietary infrastructure required.
- Economic: developable and hostable using free-tier cloud services, open-source frameworks, and free database offerings, keeping development cost negligible for an academic project.
- Operational: target users are already familiar with basic web/mobile interfaces; a simple, form-based interface minimizes the learning curve.
- Schedule: achievable within a typical single-semester academic project timeline when developed incrementally (see Section 6.4).

Appendix C: To Be Determined List

- Names and roll numbers of remaining team members (title page currently has placeholders).
- Final choice of hosting provider for the frontend, backend, and MongoDB Atlas cluster.
- Whether resolution/status notifications will be delivered by email only, or also via SMS/WhatsApp (currently listed as a future enhancement).
- Exact file-size and file-type limits for evidence image uploads.
- Formal SLA thresholds that trigger escalation of an unassigned or delayed issue.
- Academic year / semester dates for the project submission timeline.